ANÁLISIS Y DISEÑO DE ALGORITMOS

Curso 0360_2666 — Estructura de Datos y Algoritmos 2

Práctica 2

Multiplicación rápida de enteros: el algoritmo de Karatsuba

Divide y Vencerás — Recurrencias y Teorema Maestro

Informe técnico

Presentado por

Cristobal Machado Sanchez
Maria Camila Rodriguez Gonzalez
Sebastian Sanchez Gomez

1 de noviembre de 2026

1. Introducción

El método escolar para multiplicar dos enteros de n dígitos realiza $\Theta(n^2)$ productos de dígitos. En 1960 Anatolii Karatsuba mostró que, aplicando Divide y Vencerás, bastan tres multiplicaciones de números de $n/2$ dígitos en lugar de cuatro, lo que reduce el costo a $\Theta(n^{\log_2 3}) \approx \Theta(n^{1{,}585})$. Este informe deduce la identidad de Karatsuba, resuelve su recurrencia por el Teorema Maestro y por el árbol de recurrencia, describe la implementación en C++ y contrasta la teoría con mediciones de ambos algoritmos para tamaños entre 2 y 4096 dígitos.

2. Deducción de la identidad de Karatsuba

Sea B la base de numeración ($B = 10$) y $m = n/2$. Cada factor se parte en una mitad alta y una mitad baja:

$$x = x_1 B^m + x_0, \qquad y = y_1 B^m + y_0$$

Al multiplicar y distribuir se obtiene

$$x \cdot y = x_1 y_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0 \tag{1}$$

Calculado de esta forma se necesitan cuatro productos de mitades (x_1y_1, x_1y_0, x_0y_1, x_0y_0), lo que conduce a $T(n) = 4T(n/2) + \Theta(n)$ y no mejora el orden cuadrático. La observación clave es que del término central solo se necesita la suma $x_1y_0 + x_0y_1$, no cada sumando por separado, y esa suma aparece al multiplicar las sumas de las mitades:

$$(x_1 + x_0)(y_1 + y_0) = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0$$

Se definen entonces los tres productos $P_1 = x_1y_1$, $P_2 = x_0y_0$ y $P_3 = (x_1 + x_0)(y_1 + y_0)$. La igualdad anterior dice que $P_3 = P_1 + (x_1y_0 + x_0y_1) + P_2$; despejando,

$$x_1y_0 + x_0y_1 = P_3 - P_1 - P_2 \tag{2}$$

y al sustituir (2) en (1) se obtiene la identidad de Karatsuba:

$$x \cdot y = P_1 B^{2m} + (P_3 - P_1 - P_2) B^m + P_2 \tag{3}$$

Por qué tres productos y no cuatro. P_1 y P_2 ya eran dos de los cuatro productos originales; P_3 condensa en una sola multiplicación los dos productos cruzados. El precio son dos sumas previas y dos restas posteriores, operaciones $\Theta(n)$ mucho más baratas que una multiplicación de $n/2$ dígitos: se cambia una multiplicación recursiva por trabajo lineal adicional. Por ejemplo, para $12 \cdot 34$ ($B = 10$, $m = 1$): $P_1 = 1 \cdot 3 = 3$, $P_2 = 2 \cdot 4 = 8$, $P_3 = (1 + 2)(3 + 4) = 21$, el término central es $21 - 3 - 8 = 10$ y $x \cdot y = 3 \cdot 100 + 10 \cdot 10 + 8 = 408$.

De la identidad se siguen dos consecuencias para la implementación: (i) $x_1 + x_0$ e $y_1 + y_0$ pueden tener $m + 1$ dígitos, por lo que las llamadas recursivas deben admitir longitudes variables; (ii) como $P_3 - P_1 = P_2 + x_1y_0 + x_0y_1 \geq P_2$, restar primero P_1 y después P_2 nunca produce un valor negativo intermedio, y basta con aritmética sin signo.

3. Recurrencia y complejidad

Cada llamada realiza tres multiplicaciones recursivas sobre números de $n/2$ dígitos; el resto del trabajo (partir, dos sumas, dos restas, desplazamientos y recomposición) recorre los dígitos un número constante de veces, es decir, cuesta $\Theta(n)$. Por tanto,

$$T(n) = 3T(n/2) + \Theta(n), \qquad T(1) = \Theta(1) \tag{4}$$

3.1 Teorema Maestro

La recurrencia tiene la forma $T(n) = aT(n/b) + f(n)$ con $a = 3$, $b = 2$ y $f(n) = \Theta(n)$. La función de comparación es $n^{\log_b a} = n^{\log_2 3} \approx n^{1{,}585}$. Como $f(n) = \Theta(n) = O(n^{\log_2 3 - \varepsilon})$ con $\varepsilon = \log_2 3 - 1 \approx 0{,}585 > 0$, f crece polinomialmente

más despacio que $n^{\log_2 3}$. Se trata del caso 1 del teorema (dominan las hojas), de modo que $T(n) = \Theta(n^{\log_2 3})$. Como contraste, el método de cuatro productos tiene $a = 4$, $n^{\log_2 4} = n^2$ y cae en el mismo caso 1, lo que da $\Theta(n^2)$.

3.2 Árbol de recurrencia

En el nivel i hay 3^i subproblemas de tamaño $n/2^i$; cada uno realiza un trabajo no recursivo $c{\cdot}n/2^i$, de modo que el nivel completo cuesta $3^i{\cdot}c{\cdot}n/2^i = c{\cdot}n{\cdot}(3/2)^i$. El tamaño llega a 1 cuando $n/2^i = 1$, es decir, en $i = \log_2 n$: la altura es $\log_2 n$ y el último nivel tiene $3^{\log_2 n} = n^{\log_2 3}$ hojas de costo $\Theta(1)$. Sumando los niveles internos, que forman una serie geométrica de razón 3/2, y usando $(3/2)^{\log_2 n} = n^{\log_2 3}/n$:

$$T(n) = \Sigma_{i=0}^{\log_2 n - 1}\, c{\cdot}n{\cdot}(3/2)^i + \Theta(n^{\log_2 3}) = 2c{\cdot}n{\cdot}((3/2)^{\log_2 n} - 1) + \Theta(n^{\log_2 3})$$
$$= 2c{\cdot}(n^{\log_2 3} - n) + \Theta(n^{\log_2 3}) = \Theta(n^{\log_2 3}) \qquad (5)$$

3.3 Por qué el exponente es $\log_2 3$

Cada método aporta una parte de la respuesta. El Teorema Maestro clasifica la recurrencia: como $f(n)$ es polinomialmente menor que $n^{\log_2 3}$, el costo lo determinan las hojas. El árbol explica de dónde sale ese exponente: la razón $3/2 > 1$ indica que el costo por nivel crece hacia abajo, así que el trabajo total es proporcional al número de hojas, $3^{\log_2 n} = (2^{\log_2 3})^{\log_2 n} = (2^{\log_2 n})^{\log_2 3} = n^{\log_2 3}$. El 3 es el número de subproblemas y el 2 el factor de reducción del tamaño; con cuatro productos la razón sería $4/2 = 2$ y las hojas $4^{\log_2 n} = n^2$, recuperando el costo cuadrático. Eliminar un producto recursivo baja el exponente de $\log_2 4 = 2$ a $\log_2 3 \approx 1{,}585$.

Como las sumas pueden tener un dígito adicional, el subproblema real de P_3 es de tamaño $n/2 + 1$. Con el cambio de variable $S(n) = T(n + 2)$ se obtiene $S(n) = 3S(n/2) + \Theta(n)$, de modo que el orden asintótico no cambia.

4. Implementación y validación

Representación y módulos. Un número es un `std::string` de dígitos decimales con el dígito más significativo primero (tipo `bn::Num`). El archivo `bignum.hpp` contiene las operaciones auxiliares de la interfaz del enunciado (`sumar`, `restar`, `desplazar`, `partir`) más `normalizar` y `rellenar`; `multiplicacion.hpp` contiene `multiplicacionClasica` y `karatsuba`. La aritmética nativa se usa únicamente sobre dígitos individuales y acarreos.

Multiplicación clásica. Acumula el producto de cada par de dígitos en su columna dentro de un `vector<int>` y propaga los acarreos en una sola pasada final, con costo $\Theta(n^2)$.

Karatsuba. La función `karatsuba()` normaliza los factores, los rellena con ceros a la izquierda hasta una misma potencia de 2 y llama al núcleo recursivo. El núcleo iguala las longitudes de sus operandos (las sumas de P_3 pueden traer $m + 1$ dígitos), parte en $m = \lfloor n/2 \rfloor$, calcula P_1, P_2 y P_3 y recompone con $(P_3 - P_1) - P_2$. El caso base multiplica directamente cuando $n \le$ `UMBRAL_BASE` dígitos; por defecto vale 1 (Karatsuba puro) y se configura al compilar.

Validación. El programa `pruebas.cpp` ejecuta 252 532 comprobaciones, todas correctas: las operaciones auxiliares; casos de borde (un dígito, ceros a la izquierda, ceros intercalados, longitudes impares y muy distintas, factor cero) contra el valor esperado y contra la clásica; los 250 000 pares de 0 a 499 contra la aritmética nativa (usada solo como oráculo); $(10^k - 1)^2$ con su forma cerrada para $k = 1, \ldots, 1000$ en ambos algoritmos; 300 pares aleatorios de 1 a 300 dígitos y productos de 512×512, 1000×777 y 2000×2000 dígitos. Todas las pruebas se repitieron con casos base 1, 2, 3, 5, 32, 64 y 128.

5. Análisis experimental

5.1 Montaje

Las mediciones se ejecutaron en un entorno Linux virtualizado (Intel Xeon a 2,10 GHz, 2 vCPU) con g++ 13.3 (`-O2`, C++17). Los operandos son números aleatorios de *n* dígitos generados con semilla fija, con *n* = 2, 4, …, 4096. El programa `benchmark.cpp` usa `std::chrono::steady_clock`: cada tanda repite la multiplicación, duplicando el número de repeticiones hasta que dure al menos 100 ms (varios órdenes de magnitud por encima de la resolución del reloj), y se calcula el tiempo medio por multiplicación. Se realizan 5 tandas y se reporta la más rápida, que es la menos sensible al ruido del sistema. En cada repetición se cambia el último dígito de un factor y el resultado se escribe en una variable `volatile`, para que el compilador no elimine el cálculo ni lo saque del bucle; además, antes de medir cada tamaño se verifica que ambos algoritmos devuelvan el mismo producto. Se midió Karatsuba puro (caso base 1) y con casos base 32, 64 y 128; la Tabla 1 muestra la versión pura y la de caso base 64, y la columna de la clásica proviene de la corrida con caso base 1.

5.2 Resultados

Tabla 1. Tiempo medio por multiplicación. Razón = tiempo clásica / tiempo Karatsuba; en negrita, los tamaños a partir de los cuales Karatsuba supera a la clásica de forma sostenida.

***n* (dígitos)**	**Clásica (µs)**	**Karatsuba, base 1 (µs)**	**Razón (base 1)**	**Karatsuba, base 64 (µs)**	**Razón (base 64)**
2	0,04	0,62	0,058	0,11	0,341
4	0,05	2,26	0,022	0,10	0,519
8	0,10	8,99	0,011	0,16	0,617
16	0,25	28,6	0,009	0,44	0,583
32	0,65	72,6	0,009	0,85	0,766
64	2,93	221,2	0,013	2,72	1,074
128	10,9	779,7	0,014	13,1	0,829
256	28,6	2 741	0,010	46,4	0,617
512	100,9	6 296	0,016	125,0	0,807
1024	408,8	22 415	0,018	380,7	**1,074**
2048	1 598	63 968	0,025	1 288	**1,241**
4096	6 739	179 171	0,038	3 549	**1,899**

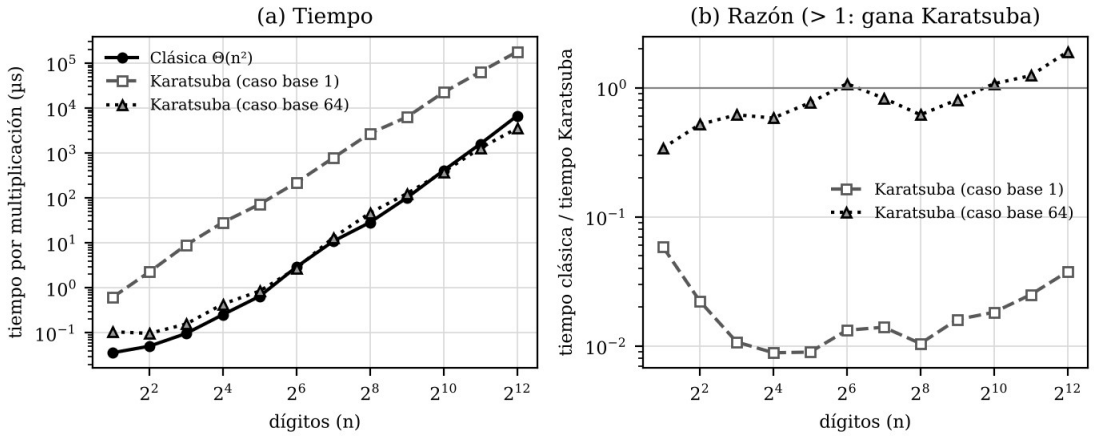


Figura 1. (a) Tiempo por multiplicación frente a n (escala log-log). (b) Razón clásica/Karatsuba; por encima de 1 gana Karatsuba.

5.3 Interpretación

Punto de cruce. Con caso base de 1 dígito, Karatsuba no supera a la clásica en el rango medido: la razón pasa de 0,009 ($n = 32$) a 0,038 ($n = 4096$), y a 4096 dígitos sigue siendo unas 27 veces más lento. Con caso base 64 el cruce aparece entre 512 y 1024 dígitos (razón 0,81 y 1,07) y a 4096 dígitos Karatsuba es 1,9 veces más rápido. El valor 1,07 en $n = 64$ no es un cruce real: para $n \le 64$ esa versión ejecuta la misma multiplicación directa, y la diferencia es ruido de medición. En las corridas adicionales el cruce quedó entre 2048 y 4096 con caso base 32 (razón 1,22 en 4096) y entre 512 y 1024 con caso base 128 (razón 2,50 en 4096). Como solo se miden potencias de 2, el cruce se acota por un intervalo, y su posición exacta depende del equipo, del compilador, de la representación y del umbral. Extrapolando la tendencia de la razón, el cruce del Karatsuba puro estaría en el orden de 10^6 a 10^7 dígitos.

Acuerdo con la teoría. Al duplicar n (para $n \ge 512$), el tiempo de la clásica se multiplica por 3,9–4,2 y el de Karatsuba puro por 2,8–3,6, cercanos a los factores 4 y 3 que predicen $\Theta(n^2)$ y $\Theta(n^{\log_2 3})$. La pendiente en escala log-log entre 512 y 4096 dígitos es 2,02 para la clásica y 1,61 para Karatsuba puro, muy próxima a $\log_2 3 \approx 1{,}585$; con caso base 64 la pendiente es también 1,61.

Por qué la clásica gana con números pequeños. La notación asintótica oculta constantes: $T_{\text{cl}}(n) \approx c_1 n^2$ frente a $T_{\text{K}}(n) \approx c_2 n^{1{,}585}$, y Karatsuba solo gana cuando $n^{0{,}415} > c_2/c_1$. En esta implementación c_2/c_1 es grande: la clásica es un doble bucle sobre memoria contigua sin asignaciones, mientras que cada nodo de Karatsuba crea más de una docena de cadenas temporales (`partir`, `rellenar`, `sumar`, `restar`, `desplazar`), con asignaciones de memoria y copias de $\Theta(n)$. Con $n = 4096$ y caso base 1 hay del orden de $3^{12} = 531\,441$ hojas, cada una con su propio vector y su cadena: el número de productos de dígitos baja de 16,8 millones a unos 0,53 millones (31 veces menos), pero el costo de gestionar los nodos supera ese ahorro. Con caso base 64 los productos son $729 \cdot 64^2 \approx 3{,}0$ millones (5,6 veces menos que la clásica) con 729 veces menos hojas, y la ventaja ya se refleja en el tiempo. En resumen, la cota asintótica describe cómo crece el costo (factores 3 y 4 por duplicación, visibles desde tamaños moderados), no cuánto vale en términos absolutos para un n dado.

6. Conclusiones y dificultades

- **Teoría.** Sustituir una multiplicación por sumas y restas lineales produce $T(n) = 3T(n/2) + \Theta(n)$, que el Teorema Maestro (caso 1) y el árbol de recurrencia resuelven en $\Theta(n^{\log_2 3})$; el exponente proviene de que el número de hojas es $3^{\log_2 n}$.
- **Práctica.** La ventaja de Karatsuba es real, pero solo aparece pasado un punto de cruce que depende de las constantes: la versión pura no lo alcanza en 4096 dígitos y la híbrida (caso base 64 o 128) lo alcanza

cerca de 1000. Por eso las implementaciones prácticas combinan Karatsuba con la multiplicación clásica en el caso base.

- **Longitudes variables.** Rellenar P_3 a la siguiente potencia de 2 en cada nivel duplicaría su tamaño (m + 1 pasa a $2m$) y destruiría la recurrencia; por eso solo se rellena a potencia de 2 en la llamada inicial y el núcleo iguala longitudes con ceros.
- **Terminación.** Con $n = 2$ o $n = 3$ el subproblema de P_3 puede conservar su tamaño (por ejemplo, 99·99 genera 18·18, de nuevo de 2 dígitos), pero en la llamada siguiente el tamaño disminuye; se comprobó con las pruebas exhaustivas y con varios casos base.
- **Resta sin signo.** El orden $(P_3 - P_1) - P_2$ evita valores negativos intermedios y simplifica `restar`, cuya precondición es $A \geq B$.
- **Medición.** Con una sola tanda el punto de cruce cambiaba entre corridas por el ruido del sistema; tomar el mínimo de 5 tandas, variar la entrada y consumir el resultado estabilizó la medición. Aun así, en tamaños donde ambos algoritmos hacen el mismo trabajo persisten diferencias de hasta un 10 %, que deben leerse como ruido. Al principio la ausencia de cruce con caso base 1 parecía un error, pero las pruebas de correctitud y el análisis de la sección 5.3 muestran que se debe al sobrecosto de la implementación.

7. Declaración de uso de inteligencia artificial

Se utilizó Claude (Anthropic) como apoyo para: proponer la estructura modular del código, las pruebas y el benchmark; verificar la terminación y la correctitud con distintos casos base; ejecutar las mediciones de la sección 5 (en el entorno descrito allí) y generar la gráfica; redactar un borrador de este informe y revisar su formato final. El grupo revisó el código y el informe y es responsable de su contenido.